Hafta 5 — Derinlemesine

> ## ⚠️ BU DOSYA SINAV KAPSAMI DIŞINDADIR
>
> Buradaki hiçbir konudan sınavda soru sorulmaz. Bu dosya, "*peki bu nasıl çalışıyor?*" diye soran öğrenci içindir. Sınav kapsamı `H05-ders-notu.md` dosyasıdır.
>
> Dönem projesinde işinize yarayacak birkaç şey de buradadır (bölüm 1, bölüm 3, bölüm 6).

---

1. Kendi Tag Helper'ınızı Yazmak

Ders notu bölüm 1.2'de Tag Helper'ın "sunucuda çalışan bir C# sınıfı" olduğunu söyledik. Burada o sınıfı yazıyoruz.

Projede hazır bir örnek var: `TagHelpers/FiyatTagHelper.cs`.

```
using Microsoft.AspNetCore.Razor.TagHelpers;

namespace KampusKitap.TagHelpers;

public class FiyatTagHelper : TagHelper
{
    public decimal Deger { get; set; }
    public string ParaBirimi { get; set; } = "₺";

    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.TagName = "span";
        output.TagMode = TagMode.StartTagAndEndTag;
        output.Attributes.SetAttribute("class", "fiyat");
        output.Content.SetContent($"{Deger:N2} {ParaBirimi}");
    }
}
```

```
<fiyat deger="@(120.50m)" />
<fiyat deger="99" para-birimi="$" />
```

```
<span class="fiyat">120,50 ₺</span>
<span class="fiyat">99,00 $</span>
```

Adlandırma kuralı

Çatı, etiket ve attribute adlarını **sınıftan türetir**:

C#	HTML
`FiyatTagHelper` sınıfı	`<fiyat>` etiketi
`Deger` özelliği	`deger` attribute'ü
`ParaBirimi` özelliği	`para-birimi` attribute'ü (**kebab-case**)

Hafta 3'te View Component'in `<vc:ziyaret-rozeti />` adını aynı kuralla aldığını görmüştük.

Kuralı beğenmezseniz açıkça yazabilirsiniz:

```
[HtmlTargetElement("para")]                    // <para> olsun
public class FiyatTagHelper : TagHelper
{
    [HtmlAttributeName("tutar")]               // attribute adı "tutar" olsun
    public decimal Deger { get; set; }
}
```

`TagHelperOutput` ile neler yapılabilir?

Yazım	Etkisi
`output.TagName = "span"`	Etiket adını değiştirir
`output.TagName = null`	Etiketi **tamamen kaldırır**, yalnızca içeriği bırakır
`output.Attributes.SetAttribute("class", "x")`	Attribute ekler/değiştirir
`output.Attributes.RemoveAll("asp-for")`	Attribute siler
`output.Content.SetContent(…)`	İçeriği **HTML olarak kodlayarak** yazar (güvenli)
`output.Content.SetHtmlContent(…)`	İçeriği **ham HTML** olarak yazar (dikkat!)
`output.PreElement.AppendHtml(…)`	Etiketin **önüne** HTML ekler
`output.PostElement.AppendHtml(…)`	Etiketin **arkasına** HTML ekler
`await output.GetChildContentAsync()`	Etiketin içindeki içeriği okur

Son maddeyi kullanmak için `Process` yerine `ProcessAsync` yazılır:

```
public override async Task ProcessAsync(TagHelperContext context, TagHelperOutput output)
{
    var icerik = await output.GetChildContentAsync();
    output.Content.SetHtmlContent("<b>" + icerik.GetContent() + "</b>");
}
```

> ⚠️ `SetHtmlContent` kullanıcıdan gelen veriyle **birleştirilmemelidir**. Hafta 3'te `@Html.Raw` için söylediğimiz XSS uyarısı burada da geçerlidir.

Var olan bir etiketi zenginleştirmek

Tag Helper yeni etiket icat etmek zorunda değildir; mevcut bir etikete **koşullu** olarak da bağlanabilir:

```
[HtmlTargetElement("a", Attributes = "dis-baglanti")]
public class DisBaglantiTagHelper : TagHelper
{
    public override void Process(TagHelperContext context, TagHelperOutput output)
    {
        output.Attributes.SetAttribute("target", "_blank");
        output.Attributes.SetAttribute("rel", "noopener noreferrer");
        output.Attributes.RemoveAll("dis-baglanti");   // kendi izini sil
    }
}
```

```
<a href="https://learn.microsoft.com" dis-baglanti>Learn</a>
```

Son satır ders notundaki teşhis kuralının sebebidir: **çalışan Tag Helper kendi attribute'ünü siler.**

Tag Helper mi, View Component mi?

Hafta 3'te View Component'i öğrenmiştik. İkisi de "parça üreten" yapılar ama işleri farklı:

	Tag Helper	View Component
Ne üretir	**Etiketin kendisini** dönüştürür	Bağımsız bir **görünüm parçası**
Kendi görünümü var mı	Hayır (kodla yazar)	**Evet** (`Default.cshtml`)
Veriyi kim getirir	Çağıran	**Kendisi** (servis enjekte edebilir)
Tipik iş	`asp-for` , `asp-action` gibi attribute'ler	Menü, rozet, sepet özeti

Kaba kural: **attribute düzeyinde bir dönüşüm** mü yapıyorsunuz → Tag Helper. **Kendi verisi olan bir blok** mu basıyorsunuz → View Component.

---

2. Çatının Diğer Hazır Tag Helper'ları

Ders notunda form ve bağlantı Tag Helper'larını gördük. Sık kullanılan birkaçı daha:

`<environment>`

Ortama göre farklı HTML üretir:

```
<environment include="Development">
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.css" />
</environment>
<environment exclude="Development">
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5/dist/css/bootstrap.min.c
</environment>
```

Geliştirmede yerel ve okunabilir dosya, yayında CDN'den küçültülmüş dosya.

`<cache>`

İçindeki parçanın çıktısını bellekte tutar:

```
<cache expires-after="@TimeSpan.FromMinutes(10)">
    @* pahalı hesaplama *@
</cache>
```

`vary-by-user`, `vary-by-route`, `vary-by-query` ile hangi koşulda ayrı kopya tutulacağı belirlenir.

`<partial>`

Hafta 3'te kullandığımız `<partial name="_KitapKart" model="k" />` de bir Tag Helper'dır — `@await Html.PartialAsync(...)` yazımının yerine geçer.

`asp-append-version`

```
<link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
```

```
<link rel="stylesheet" href="/css/site.css?v=isxqYEJw8LY-Nm8CNlRQ-pmgHVbS5O5BH-rmptf-0vI" />
```

Eklenen değer dosyanın **içeriğinin özetidir** (SHA-256). İçerik değişince adres de değişir, tarayıcı eski kopyayı kullanamaz. Buna **cache busting** denir.

> Aynı satırı `<script src="…" asp-append-version="true">` için de yazabilirsiniz. Attribute'ü **koymadığınız** dosyalarda (projedeki `bootstrap.min.css` gibi) adres sabit kalır — tarayıcı eski kopyayı günlerce kullanabilir.

---

3. `[AutoValidateAntiforgeryToken]` ve AJAX

Tek tek yazmak yerine

Ders notu bölüm 4.3'te her POST action'ına `[ValidateAntiForgeryToken]` yazdık. Unutmak mümkün — ve unutulduğunda hiçbir uyarı yok.

Daha güvenli varsayılan, **filtreyi uygulamanın tamamına** uygulamaktır:

```
builder.Services.AddControllersWithViews(secenekler =>
{
    secenekler.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
});
```

`[AutoValidateAntiforgeryToken]` **GET, HEAD, OPTIONS ve TRACE dışındaki** bütün isteklerde token arar. Yani yazmayı unuttuğunuz action da korunur.

Muaf tutmak istediğiniz bir action olursa:

```
[IgnoreAntiforgeryToken]
public IActionResult WebhookAl() { … }
```

> ⚠️ Bu ayarı **projenizin başında** yaparsanız, Hafta 4'ten kalan elle yazılmış formlarınız 400 vermeye başlar. Projede bunu bilerek yapmadık: Hafta 4'ün deneyleri çalışır durumda kalsın diye yalnızca `KitapController.EkleTH` korumaya alındı.

AJAX isteklerinde token

Form göndermeyen, JavaScript'ten POST eden sayfalarda token'ı **siz** taşırsınız. Önce sayfaya bir token bastırın:

```
@inject Microsoft.AspNetCore.Antiforgery.IAntiforgery antiforgery
@{ var token = antiforgery.GetAndStoreTokens(Context).RequestToken; }
<script>const TOKEN = "@token";</script>
```

Sonra başlıkla gönderin:

```
fetch('/Kitap/Sil/3', {
    method: 'POST',
    headers: { 'RequestVerificationToken': TOKEN }
});
```

Çatının o başlığı tanıması için:

```
builder.Services.AddAntiforgery(o => o.HeaderName = "RequestVerificationToken");
```

---

4. `SameSite` Cookie Attribute'ünün Katkısı

Ders notu bölüm 4.1'de saldırının temeli şuydu: **tarayıcı cookie'yi otomatik ekliyor.** `SameSite` attribute'ü tam olarak bu davranışı sınırlar.

`SameSite` değeri	Cookie ne zaman gönderilir
`Strict`	Yalnızca **aynı siteden** gelen isteklerde
`Lax`	Aynı siteden + **üst düzey GET gezinmelerinde** (bağlantıya tıklama)
`None`	Her zaman — ama `Secure` zorunludur

Modern tarayıcılarda varsayılan `Lax`'tır. `Lax`, başka bir siteden gelen **POST** isteğine cookie **eklemez** — yani ders notundaki saldırı senaryosunu büyük ölçüde zaten engeller.

O hâlde token'a hâlâ gerek var mı? Evet:

- `SameSite` bir **tarayıcı** korumasıdır; eski ya da ayarı değiştirilmiş bir tarayıcıda yoktur.
- Saldırı aynı siteden de gelebilir (örneğin kullanıcı içeriği barındıran bir alt alan adı).
- Güvenlik **katmanlıdır**: bir katmanın varlığı diğerini gereksiz kılmaz.

> Cookie'lerin tamamı — `Expires`, `HttpOnly`, `Secure`, `SameSite`, `Path` — **Hafta 7**'nin konusudur.

---

5. API'lerde CSRF Neden Farklı Ele Alınır?

CSRF'in çalışma şartı, kimlik bilgisinin **tarayıcı tarafından otomatik eklenmesidir**. Cookie böyledir. Ama:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs…
```

Bu başlığı tarayıcı otomatik eklemez; onu **JavaScript kodunuz** ekler. Saldırganın sayfasından yapılan istek bu başlığı taşımaz — dolayısıyla CSRF riski **doğmaz**.

Kimlik taşıyıcı	Otomatik gider mi	CSRF riski
Cookie	**Evet**	**Var** → token gerekir
`Authorization` başlığı (JWT)	Hayır	Yok

Bu yüzden Web API'lerde anti-forgery token genellikle **kullanılmaz**. Ama API'niz kimlik için **cookie** kullanıyorsa risk geri gelir.

> Web API ve JWT **Hafta 12**'nin konusudur.

---

6. Doğrulama Mesajlarını Türkçeleştirmek

Üretilen HTML'de mesajlar İngilizce:

```
data-val-required="The Başlık field is required."
```

Alan **adı** Türkçe (çünkü `[Display]` ile biz verdik), ama cümle kalıbı çatıdan geliyor. Üç yol var:

1. Mesajı attribute'te yazmak (en basit, Hafta 6'nın konusu):

```
[Required(ErrorMessage = "{0} alanı zorunludur.")]
[Display(Name = "Başlık")]
public string Baslik { get; set; } = "";
```

2. Kaynak dosyası (`.resx`) kullanmak — çok dilli projeler için doğru yol:

```
builder.Services.AddControllersWithViews()
    .AddDataAnnotationsLocalization();
```

3. Kültürü sabitlemek — ondalık ayracı sorununu da çözer:

```
var kultur = new CultureInfo("tr-TR");
app.UseRequestLocalization(new RequestLocalizationOptions
{
    DefaultRequestCulture = new RequestCulture(kultur),
    SupportedCultures = [kultur],
    SupportedUICultures = [kultur]
});
```

> Üçüncü maddenin `decimal` üzerindeki etkisi için `H04-detay.md` bölüm 5'e bakın.

---

7. `asp-for` İfadesi Nasıl Çözülüyor?

`asp-for="Baslik"` tırnak içinde **metin gibi** görünür ama değildir. Çatı onu `ModelExpression` tipinde bir ifadeye çevirir — yani derleyici onu **gerçek bir özellik erişimi** olarak görür.

Bu yüzden:

- `asp-for="Balsik"` → **derleme hatası** (`CS1061`).
- `asp-for` kullanılan görünümde `@model` yoksa model `dynamic` olur ve ifade ağacı kurulamaz → **`CS1963`**.
- İç içe erişim (`asp-for="Adres.Sehir"`) ve indis (`asp-for="Satirlar[0].Ad"`) aynı mekanizmayla çalışır.

`name` üretimi de buradan gelir: ifadenin **yolu** `name`'e yazılır. Hafta 4'te Model Binding'in `Adres.Sehir` biçimini beklediğini söylemiştik; `asp-for` tam olarak o biçimi üretir. İki hafta, aynı sözleşmenin iki ucu.

---

8. Daha İleri Gitmek İsteyenlere

Konu	Nereden başlanır
Tag Helper Components (sayfaya her görünümde içerik enjekte etmek)	`ITagHelperComponent`
`[HtmlAttributeNotBound]` ve `[ViewContext]`	Tag Helper içinden `ViewContext`'e erişmek
`Order` özelliği	Aynı etikete birden çok Tag Helper bağlandığında sıra
`asp-validation-for` / `asp-validation-summary`	**Hafta 6**
`IHtmlGenerator`	Çatının `asp-for` için kullandığı alt katman

Kaynak: Author Tag Helpers